

Laporan Teknis Komprehensif Data Science

Vitara Health.

Ekosistem Kesehatan Berbasis AI

LAPORAN TEKNIS DATASET NLP EMOTION

Dataset NLP - Emotion Classification

Laporan ini disusun untuk mendokumentasikan tahapan pengumpulan data (*data gathering*) dan preprocessing pada dataset berbasis Natural Language Processing (NLP) untuk analisis emosi dan opini publik. Dataset berasal dari repository GitHub publik bertema *emotion public opinion* dan diperkuat dengan penambahan data sintetis menggunakan Large Language Model (LLM) sebesar 30% untuk meningkatkan variasi data dan representasi emosi.

Pengerjaan dilakukan melalui dua notebook utama. Notebook pertama berfokus pada proses *data gathering*, integrasi dataset, serta penggabungan data sintetis. Notebook kedua berfokus pada preprocessing NLP yang mencakup pembersihan teks, normalisasi kata, penanganan typo, konversi emoji, analisis distribusi label, hingga finalisasi dataset siap pakai.

Hasil akhir berupa dataset NLP yang lebih bersih, konsisten, dan siap digunakan untuk analitik maupun pengembangan model machine learning.

1. Problem Discovery

Permasalahan utama dalam proyek ini adalah bagaimana membangun dataset NLP berbasis emosi dan opini publik yang bersih, konsisten, representatif, dan siap digunakan untuk analisis maupun machine learning. Beberapa tantangan utama yang ditemukan antara lain:

- Dataset berasal dari repository publik GitHub bertema *emotion public opinion* dengan struktur data dan kualitas teks yang beragam.
- Banyak ditemukan noise pada data teks seperti typo, emoji, karakter tidak relevan, singkatan informal, serta kata tidak baku (alay).
- Distribusi label emosi belum seimbang sehingga berpotensi menyebabkan bias model.
- Dataset publik memiliki keterbatasan variasi konteks bahasa sehingga diperlukan peningkatan jumlah data.
- Belum tersedia struktur data final yang siap digunakan untuk training model NLP.

Tanpa proses data gathering dan preprocessing yang baik, model machine learning berbasis NLP dapat menghasilkan performa rendah karena kualitas teks yang tidak konsisten, noisy, serta distribusi label yang tidak optimal.

2. Tujuan Proyek

Tujuan utama proyek ini adalah membangun dataset NLP yang bersih, konsisten, dan representatif untuk analisis emosi dan opini publik. Proyek ini juga bertujuan menggabungkan dataset dari GitHub dengan tambahan data sintetis dari LLM sebesar 30%, melakukan preprocessing teks secara menyeluruh, serta menghasilkan dataset akhir yang siap digunakan untuk kebutuhan machine learning dan analitik.

3. Metodologi dan Tahapan Pengerjaan

Proyek ini dilakukan melalui dua tahapan utama, yaitu data gathering dan preprocessing NLP. Tahap data gathering meliputi pengumpulan dataset dari GitHub, standardisasi struktur data, serta penambahan data sintetis menggunakan LLM sebesar 30% untuk memperkaya variasi data. Tahap preprocessing mencakup pembersihan teks, normalisasi kata tidak baku, penanganan typo, konversi emoji, analisis distribusi label, dan finalisasi dataset agar siap digunakan untuk machine learning.

4. Evidence Pengerjaan Notebook

4.1 Data Gathering

```
# Import library yang dibutuhkan
import pandas as pd
import numpy as np
import random
import hashlib

# Load dataset emosi dari GitHub public dataset
anger_url = 'https://raw.githubusercontent.com/.../AngerData.csv'
fear_url = 'https://raw.githubusercontent.com/.../FearData.csv'
joy_url = 'https://raw.githubusercontent.com/.../JoyData.csv'
sad_url = 'https://raw.githubusercontent.com/.../SadData.csv'
neutral_url = 'https://raw.githubusercontent.com/.../NeutralData.csv'

df_anger = pd.read_csv(anger_url, sep='\t')
df_fear = pd.read_csv(fear_url, sep='\t')
df_joy = pd.read_csv(joy_url, sep='\t')
df_sad = pd.read_csv(sad_url, sep='\t')
df_neutral = pd.read_csv(neutral_url, sep='\t')

# Gabungkan dataset dan standarisasi label
df_epo = pd.concat([df_anger, df_fear, df_joy, df_sad, df_neutral],
ignore_index=True)
df_epo['Label'] = df_epo['Label'].str_lower()

df_epo = df_epo.rename(columns={'Tweet': 'text', 'Label': 'label'})

# Mapping label emosi ke kategori standar
epo_map = {
    'joy': 'happy',
    'fear': 'anxious',
    'neutral': 'neutral',
    'sad': 'sad',
    'anger': 'angry'
}

df_epo['emotion_label'] = df_epo['label'].map(epo_map)
```

```

# Definisi range stress score per emosi
stress_range = {
    'happy': (0.0, 0.2),
    'neutral': (0.2, 0.4),
    'sad': (0.5, 0.8),
    'anxious': (0.7, 0.9),
    'angry': (0.85, 1.0)
}

# Generate stress score berbasis teks dan label
def generate_stress_score(text, label):
    if label not in stress_range:
        return None

    low, high = stress_range[label]

    seed = int(hashlib.md5(text.encode()).hexdigest(), 16) % (10**8)
    random.seed(seed)

    score = random.uniform(low, high)
    return round(score, 2)

# Terapkan stress score ke dataset
df_epo['stress_label'] = df_epo.apply(
    lambda x: generate_stress_score(x['text'], x['emotion_label']),
    axis=1
)

# Pilih kolom final dataset
df_epo = df_epo[['text', 'emotion_label', 'stress_label']]

# Load dataset tambahan dari LLM
df_llm =
pd.read_csv('https://raw.githubusercontent.com/.../NLP_Dataset.csv')

# Samakan label dataset LLM
df_llm['emotion_label'] = df_llm['emotion_label'].replace({'fear':
'anxious'})

# Gabungkan dataset asli dan LLM
df_combined = pd.concat([df_epo, df_llm], ignore_index=True)

# Simpan dataset akhir
df_combined.to_csv('df_combined.csv')

```

4.2 Preprocessing

```

# Install library untuk optimasi apply function
pip install swifter

# Import library yang dibutuhkan
import pandas as pd

```

```

import numpy as np
import re
import string
import swifter

# Load dataset hasil gathering (original + LLM)
df_epo =
pd.read_csv('https://raw.githubusercontent.com/.../df_epo_raw.csv')
df_llm =
pd.read_csv('https://raw.githubusercontent.com/.../df_llm_raw.csv')

# Menentukan jumlah data synthetic (30% dari dataset asli)
synthetic_count = int(len(df_epo) * 0.30)
print(f"Total synthetic used: {synthetic_count}")

# Menggabungkan dataset asli dengan data LLM sampling
df_combined = pd.concat(
    [df_epo, df_llm.sample(n=synthetic_count, random_state=42)],
    ignore_index=True
)

print(df_combined.shape)

# Balancing dataset berdasarkan jumlah kelas terbesar
max_count = df_combined['emotion_label'].value_counts().max()

balanced_data = []

for label in df_combined['emotion_label'].unique():
    subset = df_combined[df_combined['emotion_label'] == label]
    balanced_subset = subset.sample(n=max_count, replace=True,
    random_state=42)
    balanced_data.append(balanced_subset)

df_balanced = pd.concat(balanced_data, ignore_index=True)

df_balanced = df_balanced.sample(frac=1,
    random_state=42).reset_index(drop=True)

print(df_balanced.shape)
print(df_balanced['emotion_label'].value_counts())

# Copy dataset untuk preprocessing
df = df_balanced.copy()
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

# Menghapus missing value pada kolom penting
df = df.dropna(subset=['text', 'emotion_label', 'stress_label'])
df = df.reset_index(drop=True)

print(df.isnull().sum())

# Load kamus alay pertama

```

```

alay_df = pd.read_csv('https://raw.githubusercontent.com/.../colloquial-indonesian-lexicon.csv')

alay_dict = dict(zip(alay_df['slang'], alay_df['formal']))
print("Kamus alay:", len(alay_dict))

# Load kamus alay tambahan
new_alay_df = pd.read_csv(
    'https://raw.githubusercontent.com/.../new_kamusalay.csv',
    encoding='latin-1',
    names=['slang', 'formal']
)

new_alay_dict = dict(zip(new_alay_df['slang'], new_alay_df['formal']))
print("Typo dictionary:", len(new_alay_dict))

# Menggabungkan semua kamus normalisasi
combined_dict = {}
combined_dict.update(alay_dict)
combined_dict.update(new_alay_dict)

print("Total combined dictionary:", len(combined_dict))

# Kamus emoji ke teks
emoji_dict = {
    "😞": "sedih ",
    "😓": "sedih ",
    "😔": "sedih ",
    "😡": "marah ",
    "😠": "marah ",
    "😄": "senang ",
    "😁": "senang ",
    "😃": "senang ",
    "😱": "takut ",
    "😰": "cemas ",
    "😟": "cemas "
}

# Case folding (Lowercase)
def case_folding(text):
    return text.lower()

# Cleaning text (hapus noise seperti URL, angka, simbol, dll)
def clean_text(text):
    text = re.sub(r'\.[*?\\]', ' ', text)
    text = re.sub(r"http\S+", " ", text)
    text = re.sub(r"www\S+", " ", text)
    text = re.sub(r"<.*?>", " ", text)
    text = re.sub(r"@w+", " ", text)
    text = re.sub(r"#w+", " ", text)
    text = re.sub(r"d+", " ", text)

```

```

text = re.sub(r'(wkwk+|haha+|hehe+)', ' lucu ', text)
text = re.sub(r'(\1{2,})', r'\1\1', text)
text = re.sub(r'[%s]' % re.escape(string.punctuation), ' ', text)
text = re.sub(r'^a-zA-Z\s|$', ' ', text)
text = re.sub(r"\s+", " ", text)
text = text.strip()
return text

# Normalisasi kata slang ke formal
custom_normalization = {
    "gk": "tidak",
    "ga": "tidak",
    "nggak": "tidak",
    "tdk": "tidak",
    "bgt": "banget",
    "bgtt": "banget",
    "capek": "lelah",
    "cape": "lelah",
    "ngeri": "takut",
    "ovt": "cemas",
    "overthinking": "cemas",
    "stress": "cemas",
    "gw": "saya",
    "gue": "saya",
    "lu": "kamu",
    "loe": "kamu",
    "yg": "yang",
    "dr": "dari",
    "krn": "karena",
    "karna": "karena",
    "trs": "terus",
    "udh": "sudah",
    "udah": "sudah",
    "jg": "juga",
    "org": "orang"
}

# Normalisasi typo berbasis dictionary
def normalize_typo(text):
    words = text.split()
    return " ".join([custom_normalization.get(word, word) for word in words])

# Konversi emoji ke teks
def convert_emoji(text):
    for emo, meaning in emoji_dict.items():
        text = text.replace(emo, meaning)
    return text

# Normalisasi pengulangan huruf
def normalize_repeat(text):
    return re.sub(r'(\1{2,})', r'\1\1', text)

```

```

# Normalisasi kata berbasis combined dictionary
def normalize_text(text):
    words = text.split()
    return " ".join([combined_dict.get(word, word) for word in words])

# Pipeline preprocessing utama
def preprocess_text(text):
    text = str(text)
    text = case_folding(text)
    text = normalize_typo(text)
    text = convert_emoji(text)
    text = clean_text(text)
    text = normalize_repeat(text)
    text = normalize_text(text)
    return text

# Apply preprocessing menggunakan swifter (optimasi)
df['clean_text'] = df['text'].swifter.apply(preprocess_text)

print(df[['text', 'clean_text']].head())

# Hapus data kosong setelah preprocessing
df = df[df['clean_text'].str.strip() != '']
df = df.reset_index(drop=True)

# Final selection kolom dataset
df = df[['clean_text', 'emotion_label', 'stress_label']]

# Cek distribusi label
df.value_counts('emotion_label')

# Simpan dataset hasil preprocessing
df.to_csv('Preprocessed_Dataset.csv', index=False)

```

5. Data Understanding

Dataset berasal dari dua sumber utama yaitu dataset emosi publik (*emotion public opinion*) dan dataset tambahan hasil generasi LLM. Dataset ini berisi teks opini pengguna dengan label emosi seperti happy, sad, angry, anxious, dan neutral, serta fitur tambahan berupa *stress_label* yang merepresentasikan tingkat stres secara numerik.

Pada tahap ini dilakukan pemahaman struktur data, pengecekan distribusi label emosi, serta identifikasi kualitas data sebelum masuk ke tahap preprocessing. Hal ini penting untuk memastikan bahwa dataset memiliki format yang konsisten dan siap diproses lebih lanjut untuk analisis maupun machine learning.

6. Data Cleaning

Pada tahap data cleaning dilakukan proses untuk meningkatkan kualitas dataset sebelum digunakan pada model NLP. Proses ini meliputi penghapusan missing value pada kolom penting seperti *text*, *emotion_label*, dan *stress_label*, serta penghapusan data duplikat dan

data tidak valid hasil preprocessing awal.

Selain itu, dilakukan juga normalisasi struktur data agar lebih konsisten serta memastikan tidak ada nilai kosong yang dapat mengganggu proses analisis dan training model. Tahap ini bertujuan menghasilkan dataset yang lebih bersih, stabil, dan siap untuk tahap feature engineering dan modeling.

7. Feature Engineering

Pada tahap ini dilakukan transformasi dan pembersihan teks untuk meningkatkan kualitas data NLP. Proses yang dilakukan meliputi case folding, cleaning text, normalisasi kata slang, konversi emoji, penanganan repetisi huruf, serta normalisasi berbasis kamus. Hasil akhirnya adalah teks yang lebih bersih dan seragam untuk mendukung proses analisis dan modeling.

8. Distribusi Data dan Balancing

Dataset dilakukan balancing untuk mengatasi ketidakseimbangan distribusi label emosi. Selain itu, dilakukan penambahan data sintetis dari LLM sebesar 30% untuk memperkaya variasi data. Proses balancing dilakukan agar setiap kelas memiliki jumlah data yang lebih seimbang sehingga model tidak bias terhadap kelas tertentu.

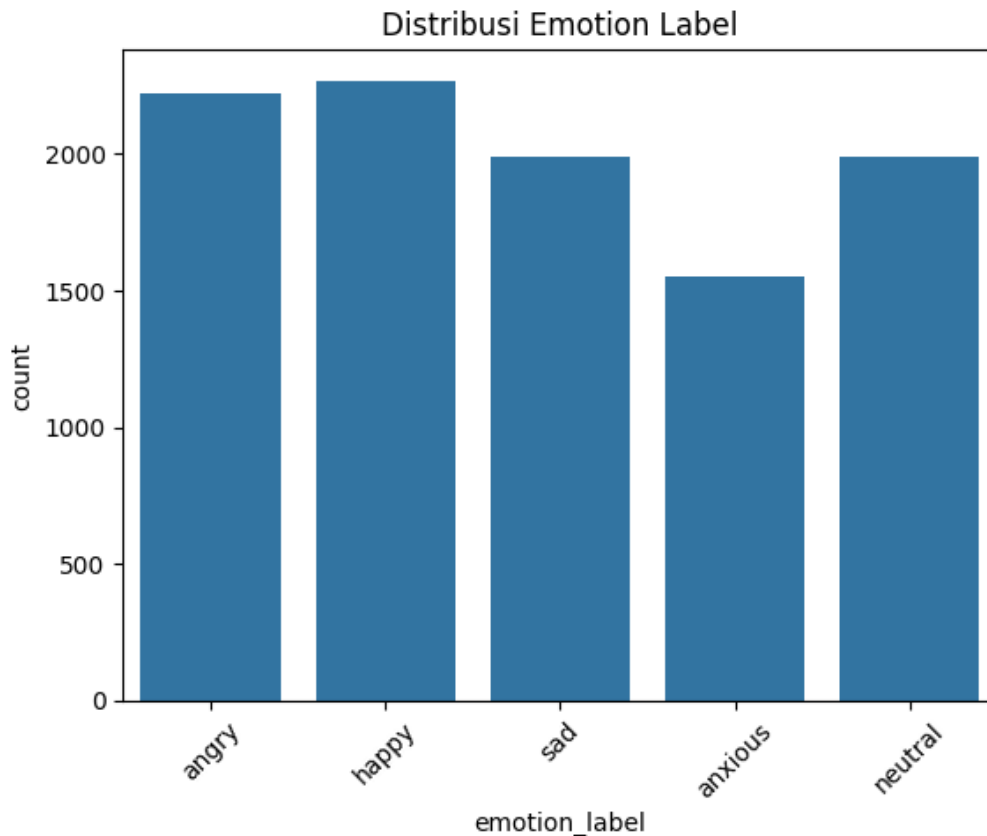
Hasil akhir distribusi label setelah balancing adalah sebagai berikut:

Emotion Label	Jumlah Data
Anxious	2371
Neutral	2371
Happy	2371
Sad	2371
Angry	2369

Distribusi ini menunjukkan bahwa dataset sudah cukup seimbang antar kelas emosi sehingga lebih ideal untuk proses training model machine learning.

9. Exploratory Data Analysis (EDA)

EDA dilakukan untuk memahami distribusi data setelah proses balancing dan preprocessing. Analisis difokuskan pada distribusi label emosi, kualitas teks, serta pemeriksaan hasil akhir dataset. Tahap ini membantu memastikan data sudah sesuai dan siap digunakan untuk modeling.



10. Final Output Dataset

Dataset akhir disimpan dalam format CSV dengan kolom utama: `clean_text`, `emotion_label`, dan `stress_label`. Dataset ini merupakan hasil dari seluruh proses data gathering, augmentasi, preprocessing, dan balancing sehingga siap digunakan untuk machine learning maupun analisis lanjutan.

Kolom	Keterangan
id	ID unik per entri
text	Teks jurnal pengguna.
emotion_label	Label emosi dominan. Nilai valid: happy, sad, anxious, angry, neutral
stress_label	Tingkat stres, skala 0.0 – 1.0
language	Kode bahasa ISO 639-1. Default: "id" (Bahasa Indonesia)

11. Kesimpulan

Proyek ini berhasil menghasilkan dataset NLP yang lebih bersih, seimbang, dan terstruktur. Seluruh tahapan mulai dari data gathering, augmentasi LLM, preprocessing teks, hingga balancing distribusi berhasil dilakukan dengan baik sehingga meningkatkan kualitas dataset secara keseluruhan.

12. Rekomendasi Action

Dataset yang dihasilkan sudah siap digunakan untuk pengembangan model NLP seperti klasifikasi emosi atau analisis opini publik. Dengan preprocessing yang baik dan distribusi data yang lebih seimbang, model diharapkan dapat menghasilkan performa yang lebih stabil dan akurat dalam memahami teks berbahasa Indonesia.

LAPORAN TEKNIS DATASET FOOD VISION

Dataset Food - Food Classification

Laporan ini disusun untuk mendokumentasikan tahapan pengumpulan data (*data gathering*) dan preprocessing pada dataset berbasis *computer vision* untuk klasifikasi gambar makanan (Food Vision). Dataset yang digunakan terdiri dari kumpulan gambar berbagai jenis makanan dengan variasi visual seperti perbedaan pencahayaan, sudut pengambilan gambar, dan bentuk makanan yang beragam.

Pengerjaan dilakukan melalui tahapan utama mulai dari pengumpulan dataset gambar, eksplorasi struktur data, hingga preprocessing gambar seperti resizing, normalisasi pixel, dan augmentasi data untuk meningkatkan variasi dataset.

Hasil akhir berupa dataset gambar makanan yang lebih terstandarisasi, konsisten, dan siap digunakan untuk analitik visual maupun pengembangan model deep learning berbasis klasifikasi gambar.

1. Problem Discovery

Permasalahan utama dalam proyek ini adalah bagaimana membangun sistem klasifikasi gambar makanan yang mampu mengenali berbagai jenis makanan secara akurat dari gambar. Tantangan yang dihadapi meliputi variasi bentuk, warna, dan pencahayaan pada gambar, serta kemiripan visual antar kelas makanan yang berbeda. Tanpa preprocessing dan model yang tepat, sistem dapat mengalami kesalahan klasifikasi akibat noise visual dan ketidakseimbangan data.

2. Tujuan Proyek

Tujuan utama proyek ini adalah membangun model klasifikasi gambar makanan yang mampu mengenali berbagai jenis makanan secara akurat. Selain itu, proyek ini bertujuan untuk menyiapkan dataset gambar yang telah melalui proses preprocessing seperti resizing, normalisasi, dan augmentasi agar lebih konsisten dan siap digunakan dalam training model deep learning.

3. Metodologi dan Tahapan Pengerjaan

Proyek ini dilakukan melalui beberapa tahapan utama mulai dari data gathering hingga preprocessing gambar. Tahap pertama adalah pengumpulan dataset gambar makanan dari sumber publik. Tahap berikutnya dilakukan eksplorasi dan pemahaman struktur dataset, termasuk distribusi kelas dan kualitas gambar.

Selanjutnya dilakukan preprocessing gambar seperti resizing, normalisasi pixel, serta augmentasi data untuk meningkatkan variasi dataset. Seluruh tahapan ini bertujuan menghasilkan dataset yang siap digunakan untuk proses training model klasifikasi gambar berbasis deep learning.

4. Evidence Pengerjaan Notebook

```
# Install dan import library yang dibutuhkan
```

```
!pip install icrawler
!pip install gdown -q
import os, shutil, random
import pandas as pd
from PIL import Image
from icrawler.builtin import BingImageCrawler
```

```
# Download dataset dari Kaggle dan Google Drive
```

```
!kaggle datasets download -d rizkyyk/dataset-food-classification
!unzip dataset-food-classification.zip
```

```
!kaggle dataset download -d dans becker/food-101
!unzip food-101.zip
```

```
!gdown https://drive.google.com/uc?id=15NakI-XT3pvJLMvUKyh3MtAMSLSTSvAV
-O mendeley.zip
!unzip -q mendeley.zip -d /content/mendeley_food
```

```
# Setup folder dataset
```

```
indo_path = 'dataset gambar/train'
food101_path = 'food-101/food-101/images'
final_path = 'data/vision/final'
extra_path = 'data/extra'
mendeley_path = '/content/mendeley_food/Indonesian Food Image/Original'
```

```
if os.path.exists(final_path):
    shutil.rmtree(final_path)
```

```
os.makedirs(final_path, exist_ok=True)
os.makedirs(extra_path, exist_ok=True)
```

```
# Copy dataset awal (Indonesia food dataset)
```

```
for cls in os.listdir(indo_path):
    if cls.startswith('.'):
        continue

    src = os.path.join(indo_path, cls)
    dst = os.path.join(final_path, cls)

    os.makedirs(dst, exist_ok=True)
```

```

    for f in os.listdir(src):
        if not f.startswith('.'):
            shutil.copy(os.path.join(src, f), dst)

# Data boosting dari Food-101
mapping_boost = {
    'Nasi Goreng': 'fried_rice',
    'Mie Goreng': 'ramen',
    'Pizza': 'pizza',
    'Burger': 'hamburger',
    'French Fries': 'french_fries',
    'Donat': 'donuts',
    'Pancake': 'pancakes'
}

MAX_PER_CLASS = 500

for indo_cls, food_cls in mapping_boost.items():
    src = os.path.join(food101_path, food_cls)
    dst = os.path.join(final_path, indo_cls)

    if not os.path.exists(src):
        continue

    os.makedirs(dst, exist_ok=True)

    existing = len(os.listdir(dst))
    needed = MAX_PER_CLASS - existing

    files = os.listdir(src)
    random.shuffle(files)

    for f in files[:needed]:
        shutil.copy(os.path.join(src, f), dst)

# Scraping gambar tambahan menggunakan Bing Image Crawler
foods = {
    'Bakso': ["bakso kuah indonesia street food real photo"],
    'Ayam Geprek': ["ayam geprek crispy indonesia sambal"],
    'Nasi Uduk': ["nasi uduk indonesia traditional food"],
    'Pecel': ["pecel sayur indonesia peanut sauce"],
    'Capcay': ["capcay indonesia stir fry vegetable"],
    'Tempe Goreng': ["tempe goreng indonesia fried tempe"]
}

```

```

}

for label, queries in foods.items():
    for q in queries:
        crawler = BingImageCrawler(
            storage={'root_dir': f'data/extra/{label}'}
        )

        crawler.crawl(keyword=q, max_num=120, min_size=(200,200))

# Merge dataset tambahan ke dataset utama
TARGET_PER_CLASS = 400

def merge_dataset(source_path):
    for cls in os.listdir(source_path):
        src = os.path.join(source_path, cls)
        dst = os.path.join(final_path, cls)

        os.makedirs(dst, exist_ok=True)

        files = os.listdir(src)
        random.shuffle(files)

        for f in files[:TARGET_PER_CLASS]:
            shutil.copy(os.path.join(src, f), dst)

# merge scraping + mendeley
merge_dataset(extra_path)
merge_dataset(mendeley_path)

# Cleaning Label mapping
mapping_fix = {
    'bakso': 'Bakso',
    'gado_gado': 'Gado-Gado',
    'nasi_goreng': 'Nasi Goreng',
    'rendang': 'Rendang',
    'rawon': 'Rawon',
    'soto': 'Soto',
    'sate': 'Sate',
    'bebek_betutu': 'Bebek Betutu',
    'pempek': 'Pempek'
}

for old, new in mapping_fix.items():

```

```

src = os.path.join(final_path, old)
dst = os.path.join(final_path, new)

if os.path.exists(src):
    os.makedirs(dst, exist_ok=True)

    for f in os.listdir(src):
        shutil.move(os.path.join(src, f), dst)

    os.rmdir(src)

# Image cleaning (filter invalid & kecil)
valid_ext = ['.jpg', '.jpeg', '.png']

def clean_folder(folder):
    for f in os.listdir(folder):
        path = os.path.join(folder, f)

        if not any(f.lower().endswith(ext) for ext in valid_ext):
            os.remove(path)
            continue

        try:
            img = Image.open(path)
            img.verify()

            if img.size[0] < 100 or img.size[1] < 100:
                os.remove(path)

        except:
            os.remove(path)

for cls in os.listdir(final_path):
    clean_folder(os.path.join(final_path, cls))

# Split dataset train/val/test
source_path = 'data/vision/final'
split_path = 'data/vision/split'

train_path = os.path.join(split_path, 'train')
val_path = os.path.join(split_path, 'val')
test_path = os.path.join(split_path, 'test')

for p in [train_path, val_path, test_path]:

```

```

os.makedirs(p, exist_ok=True)

for cls in os.listdir(source_path):
    cls_path = os.path.join(source_path, cls)

    images = os.listdir(cls_path)
    random.shuffle(images)

    train_split = int(0.8 * len(images))
    val_split = int(0.1 * len(images))

    train_files = images[:train_split]
    val_files = images[train_split:train_split + val_split]
    test_files = images[train_split + val_split:]

    for base in [train_path, val_path, test_path]:
        os.makedirs(os.path.join(base, cls), exist_ok=True)

    for f in train_files:
        shutil.copy(os.path.join(cls_path, f), os.path.join(train_path,
cls, f))

# Cek jumlah data
def count_images(folder):
    return sum(len(os.listdir(os.path.join(folder, cls))) for cls in
os.listdir(folder))

print("Train:", count_images(train_path))
print("Val:", count_images(val_path))
print("Test:", count_images(test_path))

# Zip dataset final
!zip -r dataset_food_final.zip data/vision/split

```

5. Data Understanding

Dataset yang digunakan merupakan kumpulan gambar makanan dari beberapa sumber, yaitu dataset Kaggle, Food-101, dataset Mendeley, serta hasil scraping gambar tambahan. Dataset ini mencakup berbagai kategori makanan seperti Nasi Goreng, Bakso, Rendang, Soto, Pizza, dan lainnya dengan variasi visual yang tinggi.

Pada tahap eksplorasi awal, ditemukan bahwa distribusi data antar kelas masih tidak seimbang (*imbalanced dataset*). Beberapa kelas memiliki jumlah gambar yang jauh lebih besar dibandingkan kelas lainnya. Contohnya Nasi Goreng (899 gambar) dan Sate (800

gambar), sedangkan Ayam Geprek (68 gambar) dan Capcay (79 gambar) memiliki jumlah yang jauh lebih sedikit.

Distribusi jumlah data per kelas adalah sebagai berikut:

Kelas	Jumlah
Mie Goreng	499
Nasi Goreng	899
Gudeg	400
Bakso	492
Burger	500
Ayam Goreng	400
Capcay	79
Rawon	799
Rendang	737
Nasi Uduk	84
French Fries	500
Gado-Gado	781
Donat	500
Ayam Geprek	68
Pecel	85
Bebek Betutu	399
Nasi Padang	400
Pancake	500
Soto	798
Pizza	500
Ikan Goreng	400
Pempek	399
Sate	800
Tempe Goreng	90

Karena terdapat ketidakseimbangan distribusi data, model yang digunakan yaitu **MobileNetV2** dengan pendekatan transfer learning, yang lebih robust terhadap dataset dengan ukuran tidak merata serta cocok untuk efisiensi komputasi.

6. Data Cleaning

Pada tahap ini dilakukan pembersihan dataset untuk memastikan hanya data gambar yang valid yang digunakan. Proses yang dilakukan meliputi penghapusan file non-image, file corrupt, serta gambar dengan ukuran terlalu kecil. Selain itu dilakukan standarisasi format file agar hanya menggunakan ekstensi gambar yang sesuai (.jpg, .jpeg, .png).

7. Feature Engineering

Pada proyek computer vision ini, feature engineering dilakukan dalam bentuk preprocessing gambar. Tahapan yang dilakukan meliputi resizing gambar, normalisasi pixel, serta augmentasi data untuk meningkatkan variasi dataset. Proses ini bertujuan agar model dapat lebih robust terhadap perbedaan kondisi gambar seperti cahaya, posisi, dan background.

8. Distribusi Data dan Balancing

Dataset bersifat imbalanced sehingga tidak dilakukan full balancing ketat, namun dilakukan pendekatan data enrichment melalui penggabungan multi-source dataset dan scraping tambahan. Dengan strategi ini, kelas minoritas tetap dipertahankan tetapi diperkuat melalui augmentasi data saat training.

Model yang digunakan adalah **MobileNetV2 dengan transfer learning**, yang lebih stabil terhadap kondisi dataset tidak seimbang serta efisien dalam komputasi.

9. Exploratory Data Analysis (EDA)

EDA dilakukan dengan mengecek distribusi jumlah gambar per kelas serta visualisasi sampel gambar dari setiap kategori. Dari hasil analisis terlihat bahwa terdapat perbedaan signifikan antar kelas, sehingga diperlukan strategi augmentasi saat training. Selain itu dilakukan pengecekan kualitas gambar untuk memastikan tidak ada data yang tidak valid.

10. Final Output Dataset

Dataset akhir disimpan dalam struktur folder yang siap digunakan untuk training deep learning, dengan pembagian data menjadi train, validation, dan test. Dataset kemudian diorganisir dalam format folder per kelas untuk memudahkan penggunaan pada pipeline TensorFlow.

Kolom	Keterangan
class_name	Nama folder
calories_per_100g	Kalori per 100 gram dalam kkal

11. Kesimpulan

Proyek ini berhasil membangun dataset Food Vision dari berbagai sumber dengan kualitas gambar yang sudah dibersihkan dan distandarisasi. Meskipun dataset masih bersifat tidak seimbang, data sudah cukup representatif untuk digunakan dalam training model deep learning dengan pendekatan transfer learning seperti MobileNetV2.

12. Rekomendasi Action

Dataset ini direkomendasikan untuk digunakan dengan model transfer learning seperti MobileNetV2, EfficientNet, atau ResNet. Untuk meningkatkan performa, disarankan menggunakan data augmentation yang agresif pada kelas minoritas serta fine-tuning pada layer akhir model agar hasil klasifikasi lebih akurat dan stabil pada data dunia nyata.

LAPORAN TEKNIS DATASET SLEEP LOGS

Dataset Sleep - Sleep Logs

Analisis dan Preprocessing Dataset Sleep Logs Laporan ini disusun untuk mendokumentasikan seluruh tahapan pengerjaan proyek preprocessing dan rekayasa fitur pada dataset sleep logs. Proses yang dilakukan mencakup tahapan Problem Discovery, Data Understanding, Data Cleaning, Feature Engineering, balancing distribusi data, hingga finalisasi dataset siap pakai. Notebook yang digunakan dalam proyek ini berjudul Vitara Sleep Scoring dan berfokus pada pembuatan dataset kualitas tidur yang lebih konsisten, terstruktur, dan siap digunakan untuk kebutuhan analitik maupun machine learning.

1. Problem Discovery

Permasalahan utama dalam proyek ini adalah bagaimana membangun dataset kualitas tidur yang bersih, konsisten, dan representatif untuk mendukung proses analisis pola tidur pengguna. Beberapa tantangan utama yang ditemukan antara lain: Data berasal dari beberapa sumber dataset dengan struktur yang berbeda. Terdapat kemungkinan missing value dan duplikasi data. Distribusi skor kualitas tidur tidak seimbang. Belum tersedia fitur temporal seperti tanggal tidur, jam tidur, dan wake time. Belum tersedia indikator tambahan seperti sleep debt. Tanpa proses preprocessing yang baik, model analitik maupun machine learning akan menghasilkan performa yang rendah karena kualitas data yang tidak konsisten.

2. Tujuan Proyek

Tujuan utama proyek ini adalah: Menggabungkan beberapa sumber dataset tidur menjadi satu struktur yang konsisten. Membersihkan data dari missing values dan duplikasi. Membuat fitur tambahan yang relevan dengan kualitas tidur. Membentuk distribusi data yang lebih seimbang. Menghasilkan dataset akhir yang siap digunakan untuk analisis lanjutan atau training machine learning.

3. Metodologi dan Tahapan Pengerjaan

Tahapan pengerjaan pada notebook dilakukan secara bertahap mengikuti alur preprocessing data: Import library dan loading dataset. Label mapping dan standardisasi struktur data. Data understanding. Data cleaning. Feature engineering. Distribusi dan balancing dataset. Exploratory Data Analysis (EDA). Finalisasi dan export dataset.

4. Evidence Pengerjaan Notebook

Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns from datetime import datetime, timedelta
```

Load Dataset

```
# Load data
df_stress =
pd.read_csv("https://raw.githubusercontent.com/zulhilmishodiq/Dataset/refs/heads/main/sleep_stress_dataset.csv")
df_health =
pd.read_csv("https://raw.githubusercontent.com/zulhilmishodiq/Dataset/refs/heads/main/sleep_health_dataset.csv")

print(f"Dataset Health: {df_stress.shape}")
print(f"Dataset Stress: {df_health.shape}")
```

Data Cleaning

```
# Menghapus missing values dan duplikat
df_combined = df_combined.dropna()
df_combined = df_combined.drop_duplicates()
print(f"Data Clean: {df_combined.shape}")
```

Feature Engineering

```
# Membuat Waktu Tidur & Bangun
bedtimes = []
wake_times = []
for idx, row in df_combined.iterrows():
    # Misal asumsikan waktu bangun rata-rata jam 03:00 - 09:00 pagi
    wake_hour = np.random.randint(3, 9)
    wake_minute = np.random.randint(0, 60)

    wake_dt = datetime(2026, 1, 1, wake_hour, wake_minute)
    # Bedtime = Waktu bangun dikurangi durasi tidur
    bed_dt = wake_dt - timedelta(hours=row['duration_hours'])

    bedtimes.append(bed_dt.strftime('%H:%M'))
    wake_times.append(wake_dt.strftime('%H:%M'))
```

```

df_combined['bedtime'] = bedtimes
df_combined['wake_time'] = wake_times

Balancing Data

# Undersampling
for score, target_count in target_distribution.items():

    subset = df_combined[
df_combined['quality_score'] == score
    ]
    # Kalau data lebih banyak → kurangi
    if len(subset) > target_count:
        subset = subset.sample(
n=target_count,
random_state=42
        )

    balanced_parts.append(subset)

# Gabungkan
balanced_df = pd.concat(balanced_parts)

# Acak ulang
balanced_df = balanced_df.sample(
frac=1,
random_state=42
).reset_index(drop=True)

# Mapping final
columns_order = [
    'id', 'user_id', 'date', 'bedtime', 'wake_time',
    'duration_hours', 'interruptions', 'sleep_debt_hours',
    'quality_score'
]
df_final = balanced_df[columns_order]

# Menghapus missing values dan duplikat
balanced_df = balanced_df.dropna()
balanced_df = balanced_df.drop_duplicates()

# Output dataset
print(f"Data Clean: {balanced_df.shape}")

df_final.to_csv('sleep_logs_clean.csv', index=False)

```

5. Data Understanding

Dataset berasal dari dua sumber utama yaitu dataset stress dan dataset health. Kedua dataset kemudian dilakukan proses mapping agar memiliki struktur kolom yang seragam. Kolom-kolom utama yang digunakan antara lain: `duration_hours` → durasi tidur pengguna. `interruptions` → jumlah gangguan tidur. `quality_score_raw` → skor kualitas tidur awal. `user_id` → identitas anonim pengguna. Tahapan ini penting untuk memastikan konsistensi struktur sebelum dilakukan analisis lanjutan.

6. Data Cleaning

Pada tahap data cleaning dilakukan beberapa proses penting: Menghapus missing values menggunakan `dropna()`. Menghapus data duplikat menggunakan `drop_duplicates()`. Normalisasi quality score dari skala 10 menjadi skala 0.0–1.0. Pembuatan ID unik untuk setiap record. Proses ini bertujuan meningkatkan kualitas dan reliabilitas dataset.

7. Feature Engineering

Tahap feature engineering dilakukan untuk memperkaya informasi pada dataset. Fitur tambahan yang dibuat antara lain: `Date` → sintesis tanggal tidur selama rentang tertentu. `Bedtime` → waktu mulai tidur. `Wake Time` → waktu bangun. `Sleep Debt Hours` → selisih kebutuhan tidur ideal 8 jam dengan durasi tidur aktual. Feature engineering menjadi tahap penting karena dapat meningkatkan kualitas insight dan membantu model machine learning memahami pola tidur dengan lebih baik.

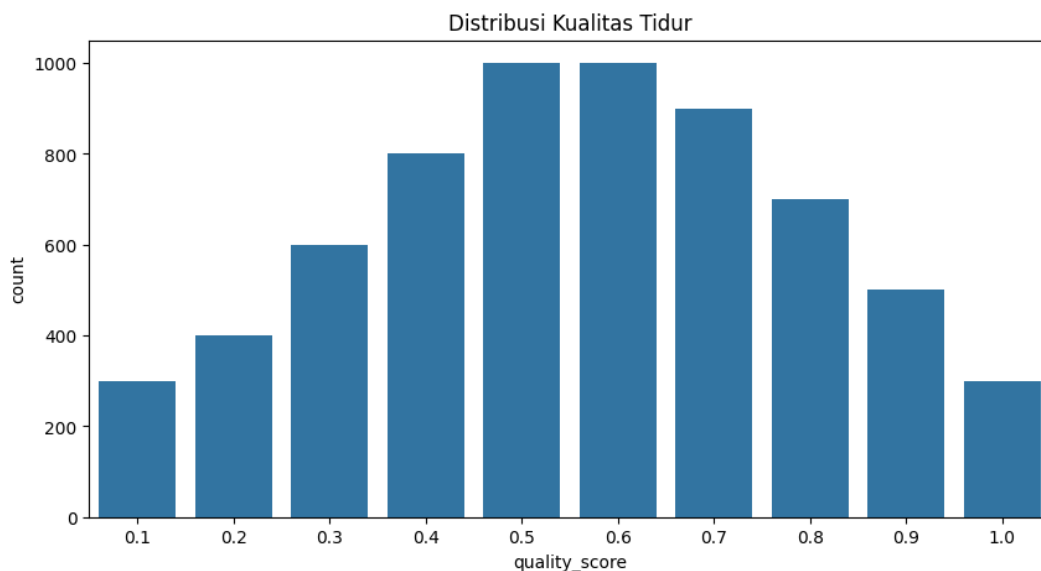
8. Distribusi Data dan Balancing

Notebook menunjukkan adanya proses balancing distribusi target quality score. Pendekatan undersampling dilakukan untuk mengurangi dominasi kelas tertentu sehingga distribusi kualitas tidur menjadi lebih proporsional. Target distribusi yang digunakan:

Score	Taerget Count
0.1	300
0.2	400
0.3	600
0.4	800
0.5	1000
0.6	1000
0.7	900
0.8	700
0.9	500
1.0	300

9. Exploratory Data Analysis (EDA)

Tahap EDA digunakan untuk memahami distribusi data akhir setelah balancing. Proses undersampling berhasil membentuk distribusi normal sehingga meminimalkan efek outlier pada `quality_score` yang ekstrem. Visualisasi countplot digunakan untuk memastikan bahwa distribusi `quality_score` sudah mendekati target distribusi yang diharapkan. EDA membantu data scientist memastikan bahwa tidak ada ketimpangan ekstrem yang dapat menyebabkan bias pada model machine learning.



10. Final Output Dataset

Dataset akhir disimpan dalam format CSV dengan nama: `sleep_logs_clean.csv`. Kolom akhir dataset: `id` `user_id` `date` `bedtime` `wake_time` `duration_hours` `interruptions` `sleep_debt_hours` `quality_score`. Dataset akhir telah berada dalam kondisi siap digunakan untuk: Machine Learning. Dashboard Analytics. Behavior Analysis. Recommendation System. Health Monitoring.

Kolom	Keterangan
id	ID unik per entri
user_id	ID anonim pengguna
date	Tanggal tidur, format YYYY-MM-DD
bedtime	Waktu mulai tidur, format HH:MM
wake_time	Waktu bangun, format HH:MM
duration_hours	Total durasi tidur dalam jam
interruptions	Jumlah kali terbangun
sleep_debt_hours	Akumulasi utang tidur dalam jam
quality_score	Label skor kualitas tidur, 0.0 -1.0

11. Kesimpulan

Preprocessing sleep logs berhasil menghasilkan dataset kualitas tidur yang lebih bersih, terstruktur, dan representatif. Seluruh tahapan preprocessing berhasil dilakukan mulai dari: Integrasi dataset multi-source. Data cleaning. Normalisasi data. Feature engineering. Balancing distribusi target. EDA dan validasi distribusi. Hasil akhir berupa dataset siap pakai yang dapat digunakan untuk modeling tim AI Engineer terkait kualitas tidur pengguna.

12. Rekomendasi Action

Dataset hasil preprocessing sudah berada dalam kondisi lebih bersih, seimbang, dan konsisten sehingga siap digunakan oleh tim AI Engineer untuk membangun Sleep Scoring Model. Feature seperti `duration_hours`, `interruptions`, dan `sleep_debt_hours` memiliki potensi kuat sebagai predictor utama dalam mengestimasi `quality_score` / kualitas tidur pengguna. Dengan distribusi target yang telah dibalancing, model diharapkan mampu menghasilkan prediksi yang lebih stabil, mengurangi bias terhadap kelas tertentu, serta meningkatkan generalisasi model pada data real-world.

LAPORAN TEKNIS DATASET KEYSTROKE

Dataset Keystroke

Laporan ini disusun untuk mendokumentasikan seluruh tahapan pengerjaan proyek preprocessing dan feature engineering pada dataset typing behavior yang digunakan dalam notebook preprocessing. Proses yang dilakukan mencakup tahapan data understanding, data cleaning, feature engineering, integrasi multi-source dataset, exploratory data analysis (EDA), hingga finalisasi dataset siap pakai.

Notebook preprocessing berfokus pada pembentukan dataset typing dynamics yang lebih konsisten, terstruktur, dan siap digunakan untuk kebutuhan analitik behavioral maupun machine learning, khususnya dalam analisis stress level berdasarkan pola pengetikan pengguna.

1. Problem Discovery

Pada tahap awal, dilakukan identifikasi terhadap permasalahan utama pada dataset typing behavior yang akan digunakan untuk kebutuhan AI modeling dan behavioral analytics. Dataset mentah masih memiliki beberapa kendala seperti struktur data yang belum konsisten, adanya missing value, duplicate records, serta kemungkinan noisy data dari aktivitas typing pengguna. Selain itu, distribusi target typing score juga belum merata sehingga berpotensi menyebabkan bias pada model machine learning.

Karakteristik data typing behavior juga cenderung sensitif terhadap anomaly dan outlier karena berkaitan langsung dengan pola interaksi pengguna terhadap keyboard. Jika dataset digunakan tanpa preprocessing yang memadai, model AI berisiko menghasilkan prediksi yang tidak stabil, overfitting terhadap pola tertentu, serta memiliki kemampuan generalisasi yang rendah pada data real-world.

Berdasarkan kondisi tersebut, preprocessing dilakukan untuk meningkatkan kualitas dataset agar lebih bersih, konsisten, dan siap digunakan pada proses modeling AI, khususnya untuk typing quality prediction dan behavioral pattern analysis.

2. Tujuan Proyek

Proyek preprocessing typing behavior ini bertujuan untuk menghasilkan dataset yang lebih bersih, konsisten, dan siap digunakan untuk kebutuhan AI modeling maupun behavioral analytics. Proses preprocessing dilakukan untuk meningkatkan kualitas data sehingga informasi yang dihasilkan lebih representatif terhadap pola interaksi pengguna saat mengetik.

Selain itu, preprocessing juga bertujuan untuk mengurangi noise, menangani missing value, serta memperbaiki distribusi data agar model machine learning dapat melakukan training secara lebih optimal. Dengan struktur dataset yang lebih stabil, model AI diharapkan mampu menghasilkan prediksi typing quality yang lebih akurat dan memiliki kemampuan generalisasi yang lebih baik pada data real-world.

3. Metodologi dan Tahapan Pengerjaan

Proses preprocessing dataset typing behavior dilakukan secara bertahap dimulai dari import library dan loading dataset, kemudian dilanjutkan dengan data understanding untuk memahami struktur data, distribusi feature, serta mengidentifikasi missing value dan anomaly pada dataset. Setelah itu dilakukan data cleaning yang mencakup handling missing value, penghapusan duplicate records, validasi tipe data, dan pembersihan noisy data untuk meningkatkan kualitas dataset. Tahap berikutnya adalah feature engineering guna membangun feature tambahan yang dapat merepresentasikan pola typing behavior pengguna secara lebih baik. Selanjutnya dilakukan evaluasi distribusi data dan balancing target untuk mengurangi bias pada model machine learning. Sebagai tahap akhir, dataset hasil preprocessing divalidasi kembali untuk memastikan data telah konsisten, stabil, dan siap digunakan pada proses AI modeling maupun behavioral analytics pipeline.

4. Evidence Pengerjaan Notebook

```
# Import Dataset

import os
import zipfile

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
from google.colab import files

# Feature Engineering

# Keyrecs

def create_inter_key_timings_keyrecs(group):
    inter_keys = group["DD"].abs().tolist()
    inter_key_string = ",".join(map(lambda x: str(int(x * 1000)),
inter_keys[:50]))

    return inter_keys, inter_key_string

# WPM Keyrecs

def calculate_wpm_keyrecs(group):
    total_chars = len(group)
    total_time_sec = (group["DD"].abs().sum())
    total_time_min = (total_time_sec / 60)

    if total_time_min == 0:
        return 0
```

```

    wpm = ((total_chars / 5) / total_time_min)

    return round(wpm, 1)

# Backspace Rate Keyrecs

def calculate_backspace_rate_keyrecs(group):
    backspace_count = ((group["key1"].astype(str).str.lower() ==
"backspace") | (group["key2"].astype(str).str.lower() == "backspace")).sum()
    total_chars = len(group)

    if total_chars == 0:
        return 0

    rate = (backspace_count / total_chars)

    return round(rate, 2)

# Aalto
# Inter Key Timings Aalto

def create_inter_key_timings_aalto(df):
    df = df.sort_values("PRESS_TIME")
    df["NEXT_PRESS"] = (df["PRESS_TIME"].shift(-1))
    df["INTER_KEY"] = (df["NEXT_PRESS"] - df["PRESS_TIME"])

    inter_keys = (df["INTER_KEY"].dropna().astype(int).tolist())
    inter_key_string = ",".join(map(str, inter_keys[:50]))

    return inter_keys, inter_key_string

# WPM Aalto

def calculate_wpm_aalto(df):
    total_chars = len(df)
    duration_ms = (df["RELEASE_TIME"].max() - df["PRESS_TIME"].min())
    duration_min = (duration_ms / 1000 / 60)

    if duration_min == 0:
        return 0

    wpm = ((total_chars / 5) / duration_min)

    return round(wpm, 1)

# Backspace Rate Aalto

def calculate_backspace_rate_aalto(df):

```

```

    backspace_count = ((df["LETTER"].astype(str).str.upper() ==
"BKSP")).sum()
    total_chars = len(df)

    if total_chars == 0:
        return 0

    rate = (backspace_count / total_chars)

    return round(rate, 2)

# Typing Variance

def calculate_typing_variance(inter_keys):
    typing_variance = np.std(inter_keys)

    return round(typing_variance, 2)

# Stress Score

def calculate_stress_score(backspace_rate, typing_variance, wpm):
    normalized_wpm = min(wpm / 120, 1)
    normalized_variance = (typing_variance / (typing_variance + 50))
    stress_score = (0.35 * backspace_rate + 0.4 * normalized_variance + 0.25
* (1 - normalized_wpm))
    stress_score = round(min(max(stress_score, 0), 1), 2)

    return stress_score

# Load Dataset

# Keyrecs dataset

df_keyrecs =
pd.read_csv("https://raw.githubusercontent.com/raihanmeintaro/Dataset/refs/heads/main/Keysroke/KeyRecs%20Dataset.csv", low_memory=False)

# Remove

df_keyrecs = df_keyrecs.loc[:, ~df_keyrecs.columns.str.contains("^Unnamed")]

# Rename

df_keyrecs.columns = [
    "participant",

```

```

    "session",
    "key1",
    "key2",
    "DU_key1_key1",
    "DD",
    "DU",
    "UD",
    "UU"
]

# Numeric

timing_cols = [
    "DU_key1_key1",
    "DD",
    "DU",
    "UD",
    "UU"
]

for col in timing_cols:
    df_keyrecs[col] = pd.to_numeric(df_keyrecs[col], errors='coerce')

df_keyrecs = df_keyrecs.dropna()

print(df_keyrecs.shape)

# Feature Engineering Keyrecs

grouped = df_keyrecs.groupby(["participant", "session"])

keyrecs_sessions = []

for idx, ((participant, session), group) in enumerate(grouped):
    if len(group) < 20:
        continue

    inter_keys, inter_key_string = (create_inter_key_timings_keyrecs(group))

    wpm = calculate_wpm_keyrecs(group)

    if wpm == 0:
        continue

    backspace_rate = (calculate_backspace_rate_keyrecs(group))
    typing_variance = (calculate_typing_variance(inter_keys))
    stress_score = (calculate_stress_score(backspace_rate, typing_variance,

```

```

wpm))

    keyrecs_sessions.append({

        "id":
        f"typ_{idx+1:05d}",

        "user_id":
        participant,

        "wpm":
        wpm,

        "typing_variance":
        typing_variance,

        "backspace_rate":
        backspace_rate,

        "inter_key_timings":
        inter_key_string,

        "stress_label":
        stress_score
    })

keyrecs_df = pd.DataFrame(keyrecs_sessions)

print(keyrecs_df.shape)
print(keyrecs_df["user_id"].nunique())

# Aalto Dataset

!wget -O Keystrokes.zip \
"https://web.archive.org/web/20251127202912if_/https://userinterfaces.aalto.fi/136Mkeystrokes/data/Keystrokes.zip"

# Extract Dataset

with zipfile.ZipFile("Keystrokes.zip", 'r') as zip_ref:
    zip_ref.extractall("aalto_dataset")

print(os.listdir("aalto_dataset"))

Load Aalto Files

DATASET_PATH = ("aalto_dataset/Keystrokes/files")

```

```

files_list = os.listdir(DATASET_PATH)

print("TOTAL FILES:", len(files_list))

# Feature Engineering Aalto

aalto_sessions = []
MAX_FILES = 2500

for idx, file in enumerate(files_list[:MAX_FILES]):
    try:
        file_path = os.path.join(DATASET_PATH, file)

        df = pd.read_csv(file_path, sep="\t", engine="python",
on_bad_lines="skip", encoding_errors="ignore")

        if len(df.columns) != 9:
            continue

        df.columns = [
            "PARTICIPANT_ID",
            "TEST_SECTION_ID",
            "SENTENCE",
            "USER_INPUT",
            "KEYSTROKE_ID",
            "PRESS_TIME",
            "RELEASE_TIME",
            "LETTER",
            "KEYCODE"
        ]

        df["PRESS_TIME"] = pd.to_numeric(df["PRESS_TIME"], errors='coerce')
        df["RELEASE_TIME"] = pd.to_numeric(df["RELEASE_TIME"],
errors='coerce')

        df = df.dropna()

        if len(df) < 20:
            continue

        inter_keys, inter_key_string = (create_inter_key_timings_aalto(df))

        if len(inter_keys) < 10:
            continue

        wpm = calculate_wpm_aalto(df)

```

```

        if wpm == 0:
            continue

        backspace_rate = (calculate_backspace_rate_aalto(df))
        typing_variance = (calculate_typing_variance(inter_keys))
        stress_score =
(calculate_stress_score(backspace_rate,typing_variance, wpm))

        aalto_sessions.append({
            "id":
            f"aalto_{idx+1:05d}",

            "user_id":
            str(
                df[
                    "PARTICIPANT_ID"
                ].iloc[0]
            ),

            "wpm":
            wpm,

            "typing_variance":
            typing_variance,

            "backspace_rate":
            backspace_rate,

            "inter_key_timings":
            inter_key_string,

            "stress_label":
            stress_score
        })

    except:
        continue

aalto_df = pd.DataFrame(aalto_sessions)

print(aalto_df.shape)
print(aalto_df["user_id"].nunique())

# Merge Real Dataset

final_df = pd.concat([keyrecs_df, aalto_df], ignore_index=True)

```

```
print(final_df.shape)
print(final_df["user_id"].nunique())

final_df.head()
```

5. Data Understanding

Dataset yang digunakan pada proyek ini berasal dari dua sumber utama yaitu Keyrecs Dataset dan Aalto Keystroke Dataset yang berisi data aktivitas pengetikan pengguna beserta informasi timing antar tombol. Kedua dataset memiliki struktur kolom yang berbeda sehingga dilakukan proses standarisasi agar dapat digabungkan dalam satu format yang konsisten. Kolom utama yang digunakan dalam analisis meliputi user_id, session_id, timing antar key event, serta informasi behavioral typing lainnya. Hasil observasi awal menunjukkan adanya perbedaan format data, kemungkinan missing values, serta belum tersedianya feature behavioral seperti inter-key timings, typing variance, backspace rate, dan stress score sehingga diperlukan proses preprocessing dan feature engineering sebelum dataset dapat digunakan untuk kebutuhan analitik maupun machine learning.

6. Data Cleaning

Pada tahap data cleaning dilakukan beberapa proses penting untuk memastikan dataset berada dalam kondisi bersih dan konsisten sebelum digunakan pada tahap analisis lanjutan maupun feature engineering. Proses cleaning meliputi penghapusan unnamed columns yang tidak memiliki kontribusi terhadap analisis, konversi kolom timing menjadi format numeric menggunakan `pd.to_numeric()`, serta penghapusan missing values menggunakan `dropna()` untuk menghindari invalid records pada proses perhitungan feature behavioral. Selain itu dilakukan juga filtering session typing yang memiliki jumlah aktivitas terlalu sedikit agar data yang digunakan lebih representatif dalam menggambarkan pola pengetikan pengguna. Tahapan data cleaning ini bertujuan meningkatkan kualitas dataset, mengurangi noise, serta memastikan seluruh feature yang dihasilkan memiliki konsistensi dan reliabilitas yang baik untuk kebutuhan machine learning maupun behavioral analysis.

7. Feature Engineering

Tahap feature engineering dilakukan untuk memperkaya informasi pada dataset typing behavior sehingga data menjadi lebih representatif dalam menggambarkan pola pengetikan pengguna. Beberapa feature behavioral yang dibuat pada notebook antara lain inter-key timings untuk mengukur jeda antar penekanan tombol, Words Per Minute (WPM) untuk menghitung kecepatan mengetik pengguna, backspace rate untuk mengidentifikasi tingkat kesalahan pengetikan, serta typing variance untuk mengukur konsistensi ritme mengetik. Selain itu dibangun juga feature stress score yang merupakan kombinasi dari beberapa indikator behavioral seperti typing variance, backspace rate, dan WPM untuk menghasilkan estimasi tingkat stress pengguna saat mengetik. Tahapan feature engineering menjadi bagian penting dalam proyek ini karena feature-feature tersebut berperan sebagai representasi behavioral typing yang nantinya dapat digunakan untuk kebutuhan machine learning, stress detection, maupun behavioral biometrics analysis.

8. Feature Engineering Dataset Keyrecs

Pada dataset Keyrecs dilakukan proses feature engineering berbasis session typing dengan pendekatan grouping berdasarkan participant dan session pengguna. Setiap session pengetikan diproses untuk menghasilkan berbagai feature behavioral seperti inter-key timings, Words Per Minute (WPM), backspace rate, typing variance, serta stress score. Inter-key timings digunakan untuk mengukur ritme antar penekanan tombol, sedangkan typing variance digunakan untuk melihat tingkat konsistensi pola mengetik pengguna. Selain itu dilakukan filtering terhadap session yang memiliki jumlah aktivitas mengetik terlalu sedikit agar data yang digunakan lebih representatif dan tidak menghasilkan noise pada proses analisis. Seluruh feature hasil engineering kemudian disimpan dalam format terstruktur sehingga dataset Keyrecs siap digabungkan dengan dataset lainnya untuk kebutuhan machine learning.

9. Feature Engineering Dataset Aalto

Pada dataset Aalto dilakukan proses feature engineering dengan pendekatan yang disesuaikan terhadap struktur data keystroke dynamics berskala besar. Dataset terlebih dahulu diunduh dan diekstrak dari file ZIP, kemudian notebook melakukan iterasi pada ribuan file session typing untuk menghasilkan feature behavioral yang seragam dengan dataset Keyrecs. Setiap session diproses untuk menghitung inter-key timings, Words Per Minute (WPM), backspace rate, typing variance, serta stress score berdasarkan aktivitas pengetikan pengguna. Selain itu dilakukan filtering terhadap data yang tidak memenuhi jumlah aktivitas minimum agar kualitas behavioral feature tetap representatif. Proses feature engineering pada dataset Aalto bertujuan memperbesar variasi pola typing behavior serta meningkatkan jumlah data training sehingga dataset akhir menjadi lebih kaya, lebih stabil, dan lebih siap digunakan untuk kebutuhan machine learning.

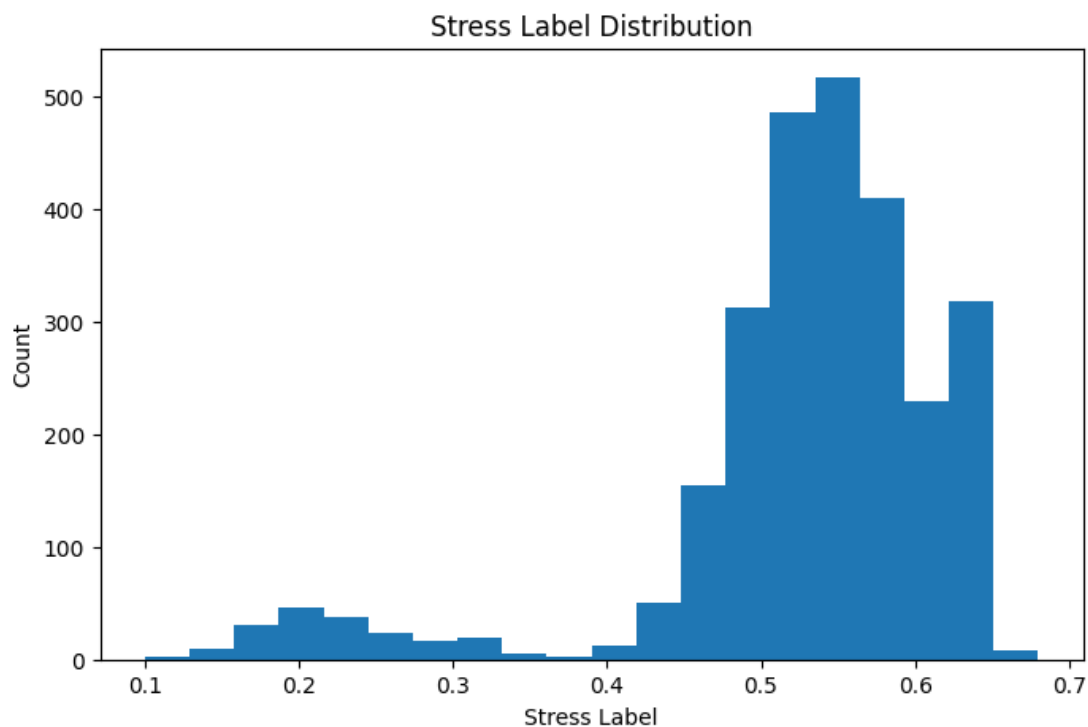
10. Merge Dataset

Setelah seluruh proses preprocessing dan feature engineering selesai dilakukan pada masing-masing dataset, notebook melakukan proses penggabungan dataset Keyrecs dan Aalto menjadi satu dataset behavioral typing yang terintegrasi. Penggabungan dilakukan menggunakan `pd.concat()` sehingga seluruh feature behavioral seperti inter-key timings, WPM, backspace rate, typing variance, dan stress score berada dalam struktur kolom yang konsisten. Tahapan merge ini bertujuan memperbesar jumlah data, meningkatkan variasi pola pengetikan pengguna, serta membentuk dataset yang lebih representatif untuk kebutuhan machine learning dan behavioral analysis. Setelah proses penggabungan selesai, dilakukan validasi jumlah data dan unique user untuk memastikan integritas dataset tetap terjaga sebelum masuk ke tahap Exploratory Data Analysis (EDA).

11. Exploratory Data Analysis (EDA)

Tahap Exploratory Data Analysis (EDA) dilakukan untuk memahami karakteristik dataset akhir setelah proses preprocessing dan penggabungan multi-source dataset selesai dilakukan. Notebook menggunakan visualisasi histogram dan distribusi stress label untuk melihat persebaran behavioral feature serta memastikan bahwa data yang dihasilkan berada dalam kondisi yang cukup representatif untuk proses modeling. Analisis dilakukan terhadap beberapa feature utama seperti WPM, typing variance, backspace rate, dan stress score guna

mengidentifikasi pola perilaku mengetik pengguna. Selain membantu memahami distribusi data, EDA juga digunakan untuk mendeteksi potensi imbalance, outlier, maupun anomali data yang dapat mempengaruhi performa machine learning model pada tahap selanjutnya.



12. Final Output Dataset

Hasil akhir dari proses preprocessing dan feature engineering adalah dataset behavioral typing yang telah berada dalam kondisi lebih bersih, konsisten, dan siap digunakan untuk kebutuhan analitik maupun machine learning. Dataset final berisi berbagai feature behavioral hasil engineering seperti inter-key timings, WPM, backspace rate, typing variance, stress score, serta stress label yang merepresentasikan pola pengetikan pengguna. Selain itu dataset telah melalui proses filtering dan validasi sehingga kualitas data menjadi lebih stabil dan representatif. Dataset akhir dapat digunakan untuk berbagai kebutuhan seperti stress detection, behavioral biometrics, user behavior analysis, human computer interaction research, serta pengembangan machine learning model berbasis typing dynamics.

Kolom	Keterangan
id	ID unik per entri
user_id	ID anonim pengguna
wpm	Kecepatan mengetik (kata per menit)
typing_variance	Variansi (Standard Deviation) dalam ms
backspace_rate	Rasio backspace (0.0 – 1.0)
inter_key_timings	Interval antar tombol dalam ms, dipisah koma
stress_label	Label tingkat stres, (0.0 – 1.0)

13. Kesimpulan

Proses preprocessing dan feature engineering pada dataset typing behavior berhasil menghasilkan dataset behavioral yang lebih bersih, konsisten, dan representatif untuk kebutuhan analitik maupun machine learning. Seluruh tahapan pengerjaan berhasil dilakukan mulai dari integrasi multi-source dataset, data cleaning, standardisasi struktur data, feature engineering behavioral, filtering session typing, hingga exploratory data analysis (EDA). Feature yang dihasilkan seperti inter-key timings, WPM, backspace rate, typing variance, dan stress score mampu memberikan representasi pola pengetikan pengguna secara lebih baik dan berpotensi digunakan sebagai indikator behavioral maupun estimasi tingkat stress. Dataset akhir telah berada dalam kondisi siap pakai sehingga dapat digunakan untuk pengembangan model stress detection, behavioral biometrics, user behavior analysis, maupun penelitian lanjutan terkait typing dynamics.

14. Rekomendasi Action

Dataset hasil preprocessing telah berada dalam kondisi lebih bersih, konsisten, dan representatif sehingga siap digunakan oleh tim AI Engineer maupun Data Scientist untuk pengembangan model behavioral analysis dan stress detection berbasis typing dynamics. Feature seperti inter-key timings, typing variance, backspace rate, serta stress score memiliki potensi kuat sebagai predictor utama dalam memahami pola perilaku mengetik pengguna. Untuk pengembangan selanjutnya disarankan dilakukan feature scaling dan feature selection guna meningkatkan performa model machine learning. Selain itu dapat ditambahkan pendekatan anomaly detection maupun sequential modeling seperti LSTM atau Transformer untuk menangkap pola temporal pada aktivitas pengetikan pengguna. Dengan dataset yang telah terstruktur dan tervalidasi, model diharapkan mampu menghasilkan prediksi yang lebih stabil, mengurangi bias data, serta meningkatkan kemampuan generalisasi terhadap data real-world.